

Python Data Structures & Algorithms Refresher

Table of Contents

1. [Introduction](#)
 2. [Environment Setup with Pixi](#)
 3. [Complexity Analysis](#)
 4. [Built-in Data Structures](#)
 5. [Advanced Data Structures](#)
 6. [Sorting Algorithms](#)
 7. [Searching Algorithms](#)
 8. [Recursion](#)
 9. [Dynamic Programming](#)
 10. [Graph Algorithms](#)
 11. [Tree Algorithms](#)
 12. [Using NumPy for Algorithms](#)
 13. [Using Pandas for Data Structures](#)
 14. [Recommended Books](#)
-

Introduction

This guide provides a comprehensive refresher on data structures and algorithms using Python. All examples use Python's standard library unless otherwise noted. Later sections cover numpy and pandas for more advanced use cases.

Key Concepts Covered:

- Time and space complexity
 - Built-in and custom data structures
 - Classic algorithms and their implementations
 - Real-world applications
-

Environment Setup with Pixi

Installing Pixi

```
# Install pixi (if not already installed)
curl -fsSL https://pixi.sh/install.sh | bash
```

Setting Up Your Project

```
# Create a new directory
mkdir python-dsa-practice
cd python-dsa-practice

# Initialize pixi project
pixi init

# Add Python
pixi add python=3.13

# For later sections, add numpy and pandas
pixi add numpy pandas
```

Running Examples

```
# Activate the environment
pixi shell

# Run your Python scripts
python your_script.py
```

Complexity Analysis

Big O Notation

Big O notation describes the upper bound of algorithm complexity.

Common Complexities:

- $O(1)$ - Constant time
- $O(\log n)$ - Logarithmic time
- $O(n)$ - Linear time
- $O(n \log n)$ - Linearithmic time
- $O(n^2)$ - Quadratic time
- $O(2^n)$ - Exponential time

```

# O(1) - Constant Time
def get_first_element(arr):
    return arr[0] if arr else None

# O(n) - Linear Time
def find_max(arr):
    max_val = arr[0]
    for num in arr:
        if num > max_val:
            max_val = num
    return max_val

# O(n²) - Quadratic Time
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# O(log n) - Logarithmic Time
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

```

Space Complexity

```

# O(1) - Constant Space
def sum_array(arr):
    total = 0
    for num in arr:
        total += num
    return total

# O(n) - Linear Space
def create_doubled_array(arr):
    return [x * 2 for x in arr]

# O(n²) - Quadratic Space
def create_multiplication_table(n):
    return [[i * j for j in range(n)] for i in range(n)]

```

Built-in Data Structures

Lists

Dynamic arrays with O(1) append and O(n) insert/delete.

```

# Creating lists
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]

# Common operations
numbers.append(6)           # O(1)
numbers.insert(0, 0)        # O(n)
numbers.pop()               # O(1)
numbers.pop(0)              # O(n)
numbers.remove(3)           # O(n)

# List comprehensions
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]

# Slicing
first_three = numbers[:3]
last_two = numbers[-2:]
reversed_list = numbers[::-1]

# Useful methods
numbers.sort()              # O(n log n)
numbers.reverse()           # O(n)
index = numbers.index(4)    # O(n)
count = numbers.count(2)    # O(n)

```

Tuples

Immutable sequences, useful for fixed collections.

```

# Creating tuples
point = (3, 4)
single = (1,) # Note the comma
empty = ()

# Unpacking
x, y = point
first, *rest = (1, 2, 3, 4, 5)

# Named tuples for clarity
from collections import namedtuple

Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p.y) # 3 4

```

Sets

Unordered collections with O(1) membership testing.

```
# Creating sets
numbers = {1, 2, 3, 4, 5}
empty_set = set()

# Operations
numbers.add(6)           # 0(1)
numbers.remove(3)        # 0(1)
numbers.discard(10)      # 0(1), no error if not found

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

union = a | b             # {1, 2, 3, 4, 5, 6}
intersection = a & b      # {3, 4}
difference = a - b        # {1, 2}
symmetric_diff = a ^ b    # {1, 2, 5, 6}

# Membership testing
print(3 in a)             # 0(1) - True
```

Dictionaries

Hash tables with O(1) average case for insert, delete, and lookup.

```
# Creating dictionaries
person = {"name": "Alice", "age": 30, "city": "NYC"}
empty_dict = {}
dict_from_pairs = dict([("a", 1), ("b", 2)])

# Operations
person["email"] = "alice@example.com" # 0(1)
age = person.get("age", 0)             # 0(1), with default
person.pop("city")                     # 0(1)

# Iteration
for key in person:
    print(key, person[key])

for key, value in person.items():
    print(f"{key}: {value}")

# Dictionary comprehension
squares = {x: x**2 for x in range(10)}

# Default dict
from collections import defaultdict

word_count = defaultdict(int)
for word in ["apple", "banana", "apple"]:
    word_count[word] += 1

# Counter
from collections import Counter

counts = Counter(["a", "b", "a", "c", "b", "a"])
print(counts) # Counter({'a': 3, 'b': 2, 'c': 1})
print(counts.most_common(2)) # [('a', 3), ('b', 2)]
```

Deque

Double-ended queue with $O(1)$ append and pop from both ends.

```
from collections import deque

# Creating deque
dq = deque([1, 2, 3])

# Operations
dq.append(4)      #  $O(1)$  - add to right
dq.appendleft(0)  #  $O(1)$  - add to left
dq.pop()          #  $O(1)$  - remove from right
dq.popleft()      #  $O(1)$  - remove from left

# Rotation
dq.rotate(1)      # Rotate right
dq.rotate(-1)     # Rotate left

# Use case: Sliding window
def max_sliding_window(nums, k):
    result = []
    window = deque()

    for i, num in enumerate(nums):
        # Remove elements outside window
        while window and window[0] <= i - k:
            window.popleft()

        # Remove smaller elements
        while window and nums[window[-1]] < num:
            window.pop()

        window.append(i)

        if i >= k - 1:
            result.append(nums[window[0]])

    return result
```

Advanced Data Structures

Stack

LIFO (Last In, First Out) structure.

```

class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        """O(1)"""
        self.items.append(item)

    def pop(self):
        """O(1)"""
        if not self.is_empty():
            return self.items.pop()
        raise IndexError("Pop from empty stack")

    def peek(self):
        """O(1)"""
        if not self.is_empty():
            return self.items[-1]
        return None

    def is_empty(self):
        """O(1)"""
        return len(self.items) == 0

    def size(self):
        """O(1)"""
        return len(self.items)

# Example: Balanced parentheses
def is_balanced(expression):
    stack = Stack()
    pairs = {'(': ')', '[': ']', '{': '}'}

    for char in expression:
        if char in pairs:
            stack.push(char)
        elif char in pairs.values():
            if stack.is_empty() or pairs[stack.pop()] != char:
                return False

    return stack.is_empty()

print(is_balanced("{[()]}" )) # True
print(is_balanced("{[(])}" )) # False

```

Queue

FIFO (First In, First Out) structure.

```

from collections import deque

class Queue:
    def __init__(self):
        self.items = deque()

    def enqueue(self, item):
        """O(1)"""
        self.items.append(item)

    def dequeue(self):
        """O(1)"""
        if not self.is_empty():
            return self.items.popleft()
        raise IndexError("Dequeue from empty queue")

    def front(self):
        """O(1)"""
        if not self.is_empty():
            return self.items[0]
        return None

    def is_empty(self):
        """O(1)"""
        return len(self.items) == 0

    def size(self):
        """O(1)"""
        return len(self.items)

# Example: BFS implementation uses queue
def bfs_level_order(root):
    if not root:
        return []

    result = []
    queue = Queue()
    queue.enqueue(root)

    while not queue.is_empty():
        level = []
        level_size = queue.size()

        for _ in range(level_size):
            node = queue.dequeue()
            level.append(node.val)

            if node.left:
                queue.enqueue(node.left)
            if node.right:
                queue.enqueue(node.right)

        result.append(level)

    return result

```

Linked List

Linear data structure with dynamic size.


```

class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class LinkedList:
    def __init__(self):
        self.head = None
        self.size = 0

    def append(self, val):
        """O(n)"""
        new_node = ListNode(val)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
        self.size += 1

    def prepend(self, val):
        """O(1)"""
        new_node = ListNode(val, self.head)
        self.head = new_node
        self.size += 1

    def delete(self, val):
        """O(n)"""
        if not self.head:
            return

        if self.head.val == val:
            self.head = self.head.next
            self.size -= 1
            return

        current = self.head
        while current.next:
            if current.next.val == val:
                current.next = current.next.next
                self.size -= 1
                return
            current = current.next

    def find(self, val):
        """O(n)"""
        current = self.head
        while current:
            if current.val == val:
                return current
            current = current.next
        return None

    def to_list(self):
        """O(n)"""
        result = []
        current = self.head
        while current:
            result.append(current.val)
            current = current.next

```

```
        return result

# Example: Reverse a linked list
def reverse_linked_list(head):
    prev = None
    current = head

    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev
```

Binary Tree

Hierarchical data structure.

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

class BinaryTree:
    def __init__(self, root=None):
        self.root = root

    def inorder(self, node, result=None):
        """Left -> Root -> Right"""
        if result is None:
            result = []
        if node:
            self.inorder(node.left, result)
            result.append(node.val)
            self.inorder(node.right, result)
        return result

    def preorder(self, node, result=None):
        """Root -> Left -> Right"""
        if result is None:
            result = []
        if node:
            result.append(node.val)
            self.preorder(node.left, result)
            self.preorder(node.right, result)
        return result

    def postorder(self, node, result=None):
        """Left -> Right -> Root"""
        if result is None:
            result = []
        if node:
            self.postorder(node.left, result)
            self.postorder(node.right, result)
            result.append(node.val)
        return result

    def height(self, node):
        """O(n)"""
        if not node:
            return 0
        return 1 + max(self.height(node.left), self.height(node.right))

# Example tree
#
#   1
#  / \
# 2   3
# / \
#4   5

root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(3)
root.left.left = TreeNode(4)
root.left.right = TreeNode(5)

tree = BinaryTree(root)
print(tree.inorder(root))    # [4, 2, 5, 1, 3]

```

```
print(tree.preorder(root)) # [1, 2, 4, 5, 3]
print(tree.postorder(root)) # [4, 5, 2, 3, 1]
```

Binary Search Tree

Binary tree with ordered property.

```

class BST:
    def __init__(self):
        self.root = None

    def insert(self, val):
        """O(log n) average, O(n) worst"""
        if not self.root:
            self.root = TreeNode(val)
        else:
            self._insert_recursive(self.root, val)

    def _insert_recursive(self, node, val):
        if val < node.val:
            if node.left:
                self._insert_recursive(node.left, val)
            else:
                node.left = TreeNode(val)
        else:
            if node.right:
                self._insert_recursive(node.right, val)
            else:
                node.right = TreeNode(val)

    def search(self, val):
        """O(log n) average, O(n) worst"""
        return self._search_recursive(self.root, val)

    def _search_recursive(self, node, val):
        if not node or node.val == val:
            return node
        if val < node.val:
            return self._search_recursive(node.left, val)
        return self._search_recursive(node.right, val)

    def delete(self, val):
        """O(log n) average, O(n) worst"""
        self.root = self._delete_recursive(self.root, val)

    def _delete_recursive(self, node, val):
        if not node:
            return None

        if val < node.val:
            node.left = self._delete_recursive(node.left, val)
        elif val > node.val:
            node.right = self._delete_recursive(node.right, val)
        else:
            # Node with one or no child
            if not node.left:
                return node.right
            if not node.right:
                return node.left

            # Node with two children
            min_node = self._find_min(node.right)
            node.val = min_node.val
            node.right = self._delete_recursive(node.right, min_node.val)

        return node

    def _find_min(self, node):
        while node.left:

```

```

        node = node.left
    return node

```

Heap (Priority Queue)

Binary heap implementation using Python's `heapq` module.

```

import heapq

class MinHeap:
    def __init__(self):
        self.heap = []

    def push(self, val):
        """O(log n)"""
        heapq.heappush(self.heap, val)

    def pop(self):
        """O(log n)"""
        if self.heap:
            return heapq.heappop(self.heap)
        raise IndexError("Pop from empty heap")

    def peek(self):
        """O(1)"""
        if self.heap:
            return self.heap[0]
        return None

    def size(self):
        """O(1)"""
        return len(self.heap)

# Max heap (negate values)
class MaxHeap:
    def __init__(self):
        self.heap = []

    def push(self, val):
        heapq.heappush(self.heap, -val)

    def pop(self):
        if self.heap:
            return -heapq.heappop(self.heap)
        raise IndexError("Pop from empty heap")

    def peek(self):
        if self.heap:
            return -self.heap[0]
        return None

# Example: Top K frequent elements
def top_k_frequent(nums, k):
    from collections import Counter

    count = Counter(nums)
    return heapq.nlargest(k, count.keys(), key=count.get)

print(top_k_frequent([1, 1, 1, 2, 2, 3], 2)) # [1, 2]

```

Graph

Adjacency list representation.

```
class Graph:
    def __init__(self, directed=False):
        self.graph = {}
        self.directed = directed

    def add_vertex(self, vertex):
        """O(1)"""
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, v1, v2):
        """O(1)"""
        self.add_vertex(v1)
        self.add_vertex(v2)
        self.graph[v1].append(v2)
        if not self.directed:
            self.graph[v2].append(v1)

    def remove_edge(self, v1, v2):
        """O(E) where E is edges from v1"""
        if v1 in self.graph and v2 in self.graph[v1]:
            self.graph[v1].remove(v2)
        if not self.directed and v2 in self.graph:
            self.graph[v2].remove(v1)

    def get_neighbors(self, vertex):
        """O(1)"""
        return self.graph.get(vertex, [])

    def __str__(self):
        return str(self.graph)

# Example usage
g = Graph()
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 3)
print(g) # {0: [1, 2], 1: [0, 2], 2: [0, 1, 3], 3: [2]}
```

Trie (Prefix Tree)

Efficient for string operations.

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        """O(m) where m is word length"""
        node = self.root
        for char in word:
            if char not in node.children:
                node.children[char] = TrieNode()
            node = node.children[char]
        node.is_end_of_word = True

    def search(self, word):
        """O(m) where m is word length"""
        node = self.root
        for char in word:
            if char not in node.children:
                return False
            node = node.children[char]
        return node.is_end_of_word

    def starts_with(self, prefix):
        """O(m) where m is prefix length"""
        node = self.root
        for char in prefix:
            if char not in node.children:
                return False
            node = node.children[char]
        return True

    def autocomplete(self, prefix):
        """Get all words with given prefix"""
        node = self.root
        for char in prefix:
            if char not in node.children:
                return []
            node = node.children[char]

        return self._find_words(node, prefix)

    def _find_words(self, node, prefix):
        words = []
        if node.is_end_of_word:
            words.append(prefix)

        for char, child_node in node.children.items():
            words.extend(self._find_words(child_node, prefix + char))

        return words

# Example
trie = Trie()
words = ["apple", "app", "application", "apply", "banana"]
for word in words:
    trie.insert(word)

```



```
print(trie.search("app"))           # True
print(trie.starts_with("app"))     # True
print(trie.autocomplete("app"))    # ['app', 'apple', 'application', 'apply']
```

Sorting Algorithms

Bubble Sort

Simple but inefficient sorting algorithm.

```
def bubble_sort(arr):
    """
    Time:  $O(n^2)$  average and worst,  $O(n)$  best
    Space:  $O(1)$ 
    Stable: Yes
    """
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

print(bubble_sort([64, 34, 25, 12, 22, 11, 90]))
```

Selection Sort

Finds minimum and swaps.

```
def selection_sort(arr):
    """
    Time:  $O(n^2)$  all cases
    Space:  $O(1)$ 
    Stable: No
    """
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

print(selection_sort([64, 25, 12, 22, 11]))
```

Insertion Sort

Builds sorted array one item at a time.

```
def insertion_sort(arr):
    """
    Time:  $O(n^2)$  average and worst,  $O(n)$  best
    Space:  $O(1)$ 
    Stable: Yes
    """
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

print(insertion_sort([12, 11, 13, 5, 6]))
```

Merge Sort

Divide and conquer algorithm.

```
def merge_sort(arr):
    """
    Time:  $O(n \log n)$  all cases
    Space:  $O(n)$ 
    Stable: Yes
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result

print(merge_sort([38, 27, 43, 3, 9, 82, 10]))
```

Quick Sort

Efficient divide and conquer algorithm.

```

def quick_sort(arr):
    """
    Time:  $O(n \log n)$  average,  $O(n^2)$  worst
    Space:  $O(\log n)$  average
    Stable: No
    """
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quick_sort(left) + middle + quick_sort(right)

# In-place version
def quick_sort_inplace(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1

    if low < high:
        pivot_idx = partition(arr, low, high)
        quick_sort_inplace(arr, low, pivot_idx - 1)
        quick_sort_inplace(arr, pivot_idx + 1, high)

    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

print(quick_sort([10, 7, 8, 9, 1, 5]))

```

Heap Sort

Uses heap data structure.

```

def heap_sort(arr):
    """
    Time: O(n log n) all cases
    Space: O(1)
    Stable: No
    """
    n = len(arr)

    # Build max heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    # Extract elements from heap
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

    return arr

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

print(heap_sort([12, 11, 13, 5, 6, 7]))

```

Counting Sort

Integer sorting algorithm.

```
def counting_sort(arr):  
    """  
    Time:  $O(n + k)$  where  $k$  is range  
    Space:  $O(k)$   
    Stable: Yes  
    Works best for small range of integers  
    """  
    if not arr:  
        return arr  
  
    min_val, max_val = min(arr), max(arr)  
    range_size = max_val - min_val + 1  
  
    count = [0] * range_size  
    output = [0] * len(arr)  
  
    # Count occurrences  
    for num in arr:  
        count[num - min_val] += 1  
  
    # Cumulative count  
    for i in range(1, len(count)):  
        count[i] += count[i - 1]  
  
    # Build output array  
    for num in reversed(arr):  
        output[count[num - min_val] - 1] = num  
        count[num - min_val] -= 1  
  
    return output  
  
print(counting_sort([4, 2, 2, 8, 3, 3, 1]))
```

Python's Built-in Sort

```
# Using sorted() - returns new list
arr = [3, 1, 4, 1, 5, 9, 2, 6]
sorted_arr = sorted(arr)           # Ascending
reverse_sorted = sorted(arr, reverse=True) # Descending

# Using list.sort() - in-place
arr.sort()
arr.sort(reverse=True)

# Custom key function
words = ["banana", "pie", "Washington", "book"]
sorted_words = sorted(words, key=len) # Sort by length
sorted_words = sorted(words, key=str.lower) # Case-insensitive

# Sorting complex objects
students = [
    {"name": "Alice", "grade": 90},
    {"name": "Bob", "grade": 85},
    {"name": "Charlie", "grade": 92}
]
sorted_students = sorted(students, key=lambda x: x["grade"], reverse=True)

# Multiple sort criteria
from operator import itemgetter
data = [(2, 3), (1, 3), (2, 1)]
sorted_data = sorted(data, key=itemgetter(1, 0)) # Sort by second, then first
```

Searching Algorithms

Linear Search

Sequential search through array.

```
def linear_search(arr, target):
    """
    Time: O(n)
    Space: O(1)
    """
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

print(linear_search([3, 1, 4, 1, 5, 9], 5)) # 4
```

Binary Search

Efficient search on sorted arrays.

```

def binary_search(arr, target):
    """
    Time: O(log n)
    Space: O(1)
    Requires: Sorted array
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Recursive version
def binary_search_recursive(arr, target, left=0, right=None):
    if right is None:
        right = len(arr) - 1

    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)

print(binary_search([1, 2, 3, 4, 5, 6, 7, 8, 9], 5)) # 4

```

Binary Search Variants

```
# Find first occurrence
def find_first(arr, target):
    left, right = 0, len(arr) - 1
    result = -1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            result = mid
            right = mid - 1 # Continue searching left
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return result

# Find last occurrence
def find_last(arr, target):
    left, right = 0, len(arr) - 1
    result = -1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            result = mid
            left = mid + 1 # Continue searching right
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return result

# Find insertion position (bisect_left)
def bisect_left(arr, target):
    left, right = 0, len(arr)

    while left < right:
        mid = (left + right) // 2
        if arr[mid] < target:
            left = mid + 1
        else:
            right = mid

    return left

print(find_first([1, 2, 2, 2, 3, 4], 2)) # 1
print(find_last([1, 2, 2, 2, 3, 4], 2)) # 3
print(bisect_left([1, 2, 4, 5], 3))    # 2
```


Jump Search

```
import math

def jump_search(arr, target):
    """
    Time: O( $\sqrt{n}$ )
    Space: O(1)
    Requires: Sorted array
    """
    n = len(arr)
    step = int(math.sqrt(n))
    prev = 0

    # Find block where element may be present
    while arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1

    # Linear search in block
    while arr[prev] < target:
        prev += 1
        if prev == min(step, n):
            return -1

    if arr[prev] == target:
        return prev

    return -1

print(jump_search([0, 1, 2, 3, 4, 5, 6, 7, 8, 9], 6)) # 6
```

Interpolation Search

```
def interpolation_search(arr, target):  
    """  
    Time: O(log log n) average, O(n) worst  
    Space: O(1)  
    Requires: Sorted array with uniform distribution  
    """  
    low, high = 0, len(arr) - 1  
  
    while low <= high and target >= arr[low] and target <= arr[high]:  
        if low == high:  
            if arr[low] == target:  
                return low  
            return -1  
  
        # Interpolation formula  
        pos = low + int(((target - arr[low]) / (arr[high] - arr[low])) * (high - low))  
  
        if arr[pos] == target:  
            return pos  
        elif arr[pos] < target:  
            low = pos + 1  
        else:  
            high = pos - 1  
  
    return -1  
  
print(interpolation_search([10, 20, 30, 40, 50], 30)) # 2
```

Recursion

Basic Recursion

```
# Factorial
def factorial(n):
    """Time: O(n), Space: O(n)"""
    if n <= 1:
        return 1
    return n * factorial(n - 1)

# Fibonacci
def fibonacci(n):
    """Time: O(2^n), Space: O(n)"""
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

# Fibonacci with memoization
def fibonacci_memo(n, memo=None):
    """Time: O(n), Space: O(n)"""
    if memo is None:
        memo = {}
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2, memo)
    return memo[n]

print(factorial(5))          # 120
print(fibonacci_memo(10))   # 55
```

Recursion with Backtracking

```
# Generate all permutations
def permutations(arr):
    """Time: O(n!), Space: O(n!)"""
    if len(arr) <= 1:
        return [arr]

    result = []
    for i in range(len(arr)):
        rest = arr[:i] + arr[i+1:]
        for p in permutations(rest):
            result.append([arr[i]] + p)

    return result

# Generate all subsets
def subsets(arr):
    """Time: O(2^n), Space: O(2^n)"""
    result = [[]]
    for num in arr:
        result += [subset + [num] for subset in result]
    return result

# Alternative recursive approach
def subsets_recursive(arr):
    if not arr:
        return [[]]

    subsets_without = subsets_recursive(arr[1:])
    subsets_with = [subset + [arr[0]] for subset in subsets_without]
    return subsets_without + subsets_with

print(permutations([1, 2, 3]))
print(subsets([1, 2, 3]))
```

Classic Recursion Problems

```
# Tower of Hanoi
def tower_of_hanoi(n, source, destination, auxiliary):
    """Time:  $O(2^n)$ , Space:  $O(n)$ """
    if n == 1:
        print(f"Move disk 1 from {source} to {destination}")
        return

    tower_of_hanoi(n - 1, source, auxiliary, destination)
    print(f"Move disk {n} from {source} to {destination}")
    tower_of_hanoi(n - 1, auxiliary, destination, source)

# N-Queens problem
def solve_n_queens(n):
    """Time:  $O(n!)$ , Space:  $O(n)$ """
    def is_safe(board, row, col):
        # Check column
        for i in range(row):
            if board[i] == col:
                return False

        # Check diagonals
        for i in range(row):
            if abs(board[i] - col) == abs(i - row):
                return False

        return True

    def backtrack(row, board):
        if row == n:
            result.append(board[:])
            return

        for col in range(n):
            if is_safe(board, row, col):
                board[row] = col
                backtrack(row + 1, board)
                board[row] = -1

    result = []
    backtrack(0, [-1] * n)
    return result

# Print solutions
solutions = solve_n_queens(4)
print(f"Found {len(solutions)} solutions for 4-Queens")
```

Dynamic Programming

Fibonacci (DP)

```
# Bottom-up approach
def fib_dp(n):
    """Time: O(n), Space: O(n)"""
    if n <= 1:
        return n

    dp = [0] * (n + 1)
    dp[1] = 1

    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]

    return dp[n]

# Space-optimized
def fib_optimized(n):
    """Time: O(n), Space: O(1)"""
    if n <= 1:
        return n

    prev2, prev1 = 0, 1

    for _ in range(2, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current

    return prev1

print(fib_dp(10))          # 55
print(fib_optimized(10))   # 55
```

Longest Common Subsequence

```
def lcs(text1, text2):
    """
    Time: O(m * n)
    Space: O(m * n)
    """
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    return dp[m][n]

# Get the actual subsequence
def lcs_with_reconstruction(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    # Reconstruct
    result = []
    i, j = m, n
    while i > 0 and j > 0:
        if text1[i - 1] == text2[j - 1]:
            result.append(text1[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] > dp[i][j - 1]:
            i -= 1
        else:
            j -= 1

    return ''.join(reversed(result))

print(lcs("ABCDGH", "AEDFHR")) # 3 (ADH)
print(lcs_with_reconstruction("ABCDGH", "AEDFHR")) # "ADH"
```

Knapsack Problem

```
def knapsack_01(weights, values, capacity):
    """
    0/1 Knapsack Problem
    Time: O(n * capacity)
    Space: O(n * capacity)
    """
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

# Space-optimized version
def knapsack_optimized(weights, values, capacity):
    """Time: O(n * capacity), Space: O(capacity)"""
    dp = [0] * (capacity + 1)

    for i in range(len(weights)):
        for w in range(capacity, weights[i] - 1, -1):
            dp[w] = max(dp[w], values[i] + dp[w - weights[i]])

    return dp[capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print(knapsack_01(weights, values, capacity)) # 9
```


Coin Change

```
# Minimum coins needed
def coin_change_min(coins, amount):
    """
    Time: O(amount * n)
    Space: O(amount)
    """
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0

    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] = min(dp[i], dp[i - coin] + 1)

    return dp[amount] if dp[amount] != float('inf') else -1

# Number of ways to make change
def coin_change_ways(coins, amount):
    """
    Time: O(amount * n)
    Space: O(amount)
    """
    dp = [0] * (amount + 1)
    dp[0] = 1

    for coin in coins:
        for i in range(coin, amount + 1):
            dp[i] += dp[i - coin]

    return dp[amount]

print(coin_change_min([1, 2, 5], 11)) # 3 (5+5+1)
print(coin_change_ways([1, 2, 5], 5)) # 4 ways
```

Longest Increasing Subsequence

```
def longest_increasing_subsequence(nums):
    """
    Time: O(n²)
    Space: O(n)
    """
    if not nums:
        return 0

    n = len(nums)
    dp = [1] * n

    for i in range(1, n):
        for j in range(i):
            if nums[j] < nums[i]:
                dp[i] = max(dp[i], dp[j] + 1)

    return max(dp)

# Optimized O(n log n) solution
def lis_optimized(nums):
    """
    Time: O(n log n)
    Space: O(n)
    """
    from bisect import bisect_left

    if not nums:
        return 0

    tails = []

    for num in nums:
        pos = bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num

    return len(tails)

print(longest_increasing_subsequence([10, 9, 2, 5, 3, 7, 101, 18])) # 4
```

Edit Distance

```
def edit_distance(word1, word2):  
    """  
    Levenshtein distance  
    Time: O(m * n)  
    Space: O(m * n)  
    """  
  
    m, n = len(word1), len(word2)  
    dp = [[0] * (n + 1) for _ in range(m + 1)]  
  
    # Initialize base cases  
    for i in range(m + 1):  
        dp[i][0] = i  
    for j in range(n + 1):  
        dp[0][j] = j  
  
    # Fill the table  
    for i in range(1, m + 1):  
        for j in range(1, n + 1):  
            if word1[i - 1] == word2[j - 1]:  
                dp[i][j] = dp[i - 1][j - 1]  
            else:  
                dp[i][j] = 1 + min(  
                    dp[i - 1][j],      # Delete  
                    dp[i][j - 1],      # Insert  
                    dp[i - 1][j - 1]    # Replace  
                )  
  
    return dp[m][n]  
  
print(edit_distance("horse", "ros")) # 3  
print(edit_distance("intention", "execution")) # 5
```

Graph Algorithms

Depth-First Search (DFS)

```
def dfs_recursive(graph, start, visited=None):
    """
    Time:  $O(V + E)$ 
    Space:  $O(V)$ 
    """
    if visited is None:
        visited = set()

    visited.add(start)
    print(start, end=' ')

    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

    return visited

def dfs_iterative(graph, start):
    """
    Time:  $O(V + E)$ 
    Space:  $O(V)$ 
    """
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            visited.add(vertex)
            print(vertex, end=' ')

            # Add neighbors in reverse to match recursive order
            for neighbor in reversed(graph.get(vertex, [])):
                if neighbor not in visited:
                    stack.append(neighbor)

    return visited

graph = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [3]
}

print("\nDFS Recursive:", end=' ')
dfs_recursive(graph, 2)
print("\nDFS Iterative:", end=' ')
dfs_iterative(graph, 2)
```

Breadth-First Search (BFS)

```

from collections import deque

def bfs(graph, start):
    """
    Time:  $O(V + E)$ 
    Space:  $O(V)$ 
    """
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        vertex = queue.popleft()
        result.append(vertex)

        for neighbor in graph.get(vertex, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return result

# BFS with level tracking
def bfs_levels(graph, start):
    """Returns nodes at each level"""
    visited = set([start])
    queue = deque([(start, 0)])
    levels = {}

    while queue:
        vertex, level = queue.popleft()
        if level not in levels:
            levels[level] = []
        levels[level].append(vertex)

        for neighbor in graph.get(vertex, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, level + 1))

    return levels

print("\nBFS:", bfs(graph, 2))
print("BFS Levels:", bfs_levels(graph, 2))

```

Shortest Path (Dijkstra's Algorithm)

```

import heapq

def dijkstra(graph, start):
    """
    Time:  $O((V + E) \log V)$ 
    Space:  $O(V)$ 
    Graph format: {vertex: [(neighbor, weight), ...]}
    """
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0
    pq = [(0, start)]

    while pq:
        current_dist, current = heapq.heappop(pq)

        if current_dist > distances[current]:
            continue

        for neighbor, weight in graph.get(current, []):
            distance = current_dist + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return distances

# With path reconstruction
def dijkstra_with_path(graph, start, end):
    distances = {vertex: float('inf') for vertex in graph}
    distances[start] = 0
    previous = {vertex: None for vertex in graph}
    pq = [(0, start)]

    while pq:
        current_dist, current = heapq.heappop(pq)

        if current == end:
            break

        if current_dist > distances[current]:
            continue

        for neighbor, weight in graph.get(current, []):
            distance = current_dist + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                previous[neighbor] = current
                heapq.heappush(pq, (distance, neighbor))

    # Reconstruct path
    path = []
    current = end
    while current is not None:
        path.append(current)
        current = previous[current]
    path.reverse()

    return distances[end], path

weighted_graph = {

```

```
'A': [('B', 4), ('C', 2)],  
'B': [('C', 1), ('D', 5)],  
'C': [('D', 8), ('E', 10)],  
'D': [('E', 2)],  
'E': []  
}  
  
print("\nDijkstra from A:", dijkstra(weighted_graph, 'A'))  
print("Path A to E:", dijkstra_with_path(weighted_graph, 'A', 'E'))
```


Topological Sort

```

def topological_sort_dfs(graph):
    """
    Time:  $O(V + E)$ 
    Space:  $O(V)$ 
    For Directed Acyclic Graph (DAG)
    """
    visited = set()
    stack = []

    def dfs(vertex):
        visited.add(vertex)
        for neighbor in graph.get(vertex, []):
            if neighbor not in visited:
                dfs(neighbor)
        stack.append(vertex)

    for vertex in graph:
        if vertex not in visited:
            dfs(vertex)

    return stack[::-1]

# Kahn's algorithm (BFS-based)
def topological_sort_kahn(graph):
    """
    Time:  $O(V + E)$ 
    Space:  $O(V)$ 
    """
    from collections import defaultdict, deque

    # Calculate in-degrees
    in_degree = defaultdict(int)
    for vertex in graph:
        for neighbor in graph[vertex]:
            in_degree[neighbor] += 1

    # Queue with vertices having no incoming edges
    queue = deque([v for v in graph if in_degree[v] == 0])
    result = []

    while queue:
        vertex = queue.popleft()
        result.append(vertex)

        for neighbor in graph.get(vertex, []):
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    # Check for cycle
    if len(result) != len(graph):
        return None # Graph has a cycle

    return result

dag = {
    5: [2, 0],
    4: [0, 1],
    2: [3],
    3: [1],
    1: [],
    0: []
}

```

```

}

print("\nTopological Sort (DFS):", topological_sort_dfs(dag))
print("Topological Sort (Kahn):", topological_sort_kahn(dag))

```

Cycle Detection

```

def has_cycle_directed(graph):
    """Detect cycle in directed graph"""
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {vertex: WHITE for vertex in graph}

    def dfs(vertex):
        color[vertex] = GRAY

        for neighbor in graph.get(vertex, []):
            if color[neighbor] == GRAY:
                return True # Back edge found
            if color[neighbor] == WHITE and dfs(neighbor):
                return True

        color[vertex] = BLACK
        return False

    for vertex in graph:
        if color[vertex] == WHITE:
            if dfs(vertex):
                return True

    return False

def has_cycle_undirected(graph):
    """Detect cycle in undirected graph"""
    visited = set()

    def dfs(vertex, parent):
        visited.add(vertex)

        for neighbor in graph.get(vertex, []):
            if neighbor not in visited:
                if dfs(neighbor, vertex):
                    return True
            elif neighbor != parent:
                return True

        return False

    for vertex in graph:
        if vertex not in visited:
            if dfs(vertex, None):
                return True

    return False

cyclic_graph = {0: [1], 1: [2], 2: [0]}
acyclic_graph = {0: [1], 1: [2], 2: []}

print("\nCyclic graph has cycle:", has_cycle_directed(cyclic_graph))
print("Acyclic graph has cycle:", has_cycle_directed(acyclic_graph))

```

Union-Find (Disjoint Set)

```
class UnionFind:
    """
    Used for cycle detection and connected components
    """
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        """Find with path compression -  $O(\alpha(n))$ """
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x, y):
        """Union by rank -  $O(\alpha(n))$ """
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return False # Already in same set

        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        elif self.rank[root_x] > self.rank[root_y]:
            self.parent[root_y] = root_x
        else:
            self.parent[root_y] = root_x
            self.rank[root_x] += 1

        return True

    def connected(self, x, y):
        """Check if two elements are in same set"""
        return self.find(x) == self.find(y)

# Kruskal's Minimum Spanning Tree
def kruskal_mst(n, edges):
    """
    Time:  $O(E \log E)$ 
    edges: [(weight, u, v), ...]
    """
    uf = UnionFind(n)
    edges.sort() # Sort by weight
    mst = []
    total_weight = 0

    for weight, u, v in edges:
        if uf.union(u, v):
            mst.append((u, v, weight))
            total_weight += weight

    return mst, total_weight

edges = [(1, 0, 1), (2, 0, 2), (3, 1, 2), (4, 1, 3), (5, 2, 3)]
print("\nMST:", kruskal_mst(4, edges))
```

Tree Algorithms

Tree Traversals

```

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

```

```

# Iterative traversals

```

```

def inorder_iterative(root):
    """Left -> Root -> Right"""
    result = []
    stack = []
    current = root

    while current or stack:
        while current:
            stack.append(current)
            current = current.left

        current = stack.pop()
        result.append(current.val)
        current = current.right

    return result

def preorder_iterative(root):
    """Root -> Left -> Right"""
    if not root:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.val)

        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)

    return result

def postorder_iterative(root):
    """Left -> Right -> Root"""
    if not root:
        return []

    result = []
    stack = [root]

    while stack:
        node = stack.pop()
        result.append(node.val)

        if node.left:
            stack.append(node.left)
        if node.right:
            stack.append(node.right)

    return result[::-1]

```

```
def level_order(root):  
    """BFS level by level"""  
    if not root:  
        return []  
  
    result = []  
    queue = deque([root])  
  
    while queue:  
        level = []  
        for _ in range(len(queue)):   
            node = queue.popleft()  
            level.append(node.val)  
  
            if node.left:  
                queue.append(node.left)  
            if node.right:  
                queue.append(node.right)  
  
        result.append(level)  
  
    return result
```

Common Tree Problems


```

def max_depth(root):
    """Maximum depth of binary tree"""
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))

def is_balanced(root):
    """Check if tree is height-balanced"""
    def height(node):
        if not node:
            return 0

        left_height = height(node.left)
        if left_height == -1:
            return -1

        right_height = height(node.right)
        if right_height == -1:
            return -1

        if abs(left_height - right_height) > 1:
            return -1

        return 1 + max(left_height, right_height)

    return height(root) != -1

def lowest_common_ancestor(root, p, q):
    """Find LCA of two nodes"""
    if not root or root == p or root == q:
        return root

    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)

    if left and right:
        return root

    return left if left else right

def is_valid_bst(root):
    """Validate binary search tree"""
    def validate(node, low=float('-inf'), high=float('inf')):
        if not node:
            return True

        if node.val <= low or node.val >= high:
            return False

        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))

    return validate(root)

def serialize_tree(root):
    """Serialize tree to string"""
    def helper(node):
        if not node:
            vals.append('#')
            return
        vals.append(str(node.val))
        helper(node.left)

```

```
        helper(node.right)

    vals = []
    helper(root)
    return ','.join(vals)

def deserialize_tree(data):
    """Deserialize string to tree"""
    def helper():
        val = next(vals)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = helper()
        node.right = helper()
        return node

    vals = iter(data.split(','))
    return helper()
```

Path Sum Problems

```
def has_path_sum(root, target_sum):
    """Check if any root-to-leaf path sums to target"""
    if not root:
        return False

    if not root.left and not root.right:
        return root.val == target_sum

    remaining = target_sum - root.val
    return (has_path_sum(root.left, remaining) or
            has_path_sum(root.right, remaining))

def path_sum_all(root, target_sum):
    """Find all root-to-leaf paths that sum to target"""
    def dfs(node, remaining, path, result):
        if not node:
            return

        path.append(node.val)

        if not node.left and not node.right and remaining == node.val:
            result.append(path[:])

        dfs(node.left, remaining - node.val, path, result)
        dfs(node.right, remaining - node.val, path, result)

        path.pop()

    result = []
    dfs(root, target_sum, [], result)
    return result

def max_path_sum(root):
    """Maximum path sum (any node to any node)"""
    def helper(node):
        nonlocal max_sum
        if not node:
            return 0

        left = max(0, helper(node.left))
        right = max(0, helper(node.right))

        max_sum = max(max_sum, node.val + left + right)

        return node.val + max(left, right)

    max_sum = float('-inf')
    helper(root)
    return max_sum
```

Using NumPy for Algorithms

NumPy provides efficient array operations and mathematical functions.

Installation

```
pixi add numpy
```

Arrays and Basic Operations

```
import numpy as np

# Creating arrays
arr1d = np.array([1, 2, 3, 4, 5])
arr2d = np.array([[1, 2, 3], [4, 5, 6]])
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
identity = np.eye(3)
random_arr = np.random.rand(3, 3)

# Array attributes
print(arr2d.shape)      # (2, 3)
print(arr2d.dtype)      # int64
print(arr2d.size)       # 6
print(arr2d.ndim)       # 2

# Reshaping
reshaped = np.arange(12).reshape(3, 4)
flattened = reshaped.flatten()
transposed = reshaped.T

# Indexing and slicing
print(arr2d[0, 1])      # 2
print(arr2d[:, 1])       # [2, 5]
print(arr2d[0, :])      # [1, 2, 3]

# Boolean indexing
mask = arr1d > 2
print(arr1d[mask])      # [3, 4, 5]
```

Mathematical Operations

```
# Element-wise operations
a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

print(a + b)          # [6, 8, 10, 12]
print(a * b)          # [5, 12, 21, 32]
print(a ** 2)         # [1, 4, 9, 16]

# Universal functions
print(np.sqrt(a))
print(np.exp(a))
print(np.log(a))
print(np.sin(a))

# Aggregations
print(np.sum(a))      # 10
print(np.mean(a))     # 2.5
print(np.std(a))      # 1.118
print(np.min(a))      # 1
print(np.max(a))      # 4

# Matrix operations
matrix_a = np.array([[1, 2], [3, 4]])
matrix_b = np.array([[5, 6], [7, 8]])

print(np.dot(matrix_a, matrix_b))    # Matrix multiplication
print(matrix_a @ matrix_b)           # Alternative syntax
print(np.linalg.inv(matrix_a))       # Inverse
print(np.linalg.det(matrix_a))       # Determinant
```

Algorithmic Applications

```

# Efficient array searching
def binary_search_numpy(arr, target):
    """Using numpy's searchsorted"""
    idx = np.searchsorted(arr, target)
    if idx < len(arr) and arr[idx] == target:
        return idx
    return -1

# Computing distances (useful for clustering)
def euclidean_distance_matrix(points):
    """Compute pairwise distances between points"""
    # points: (n, d) array where n is number of points, d is dimensions
    diff = points[:, np.newaxis, :] - points[np.newaxis, :, :]
    distances = np.sqrt(np.sum(diff ** 2, axis=-1))
    return distances

# Example
points = np.array([[0, 0], [1, 1], [2, 2]])
print(euclidean_distance_matrix(points))

# Moving average (sliding window)
def moving_average(arr, window_size):
    """Compute moving average efficiently"""
    cumsum = np.cumsum(np.insert(arr, 0, 0))
    return (cumsum[window_size:] - cumsum[:-window_size]) / window_size

data = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(moving_average(data, 3))

# Finding top K elements
def top_k_elements(arr, k):
    """Get indices and values of top k elements"""
    indices = np.argpartition(arr, -k)[-k:]
    indices = indices[np.argsort(arr[indices])][::-1]
    return indices, arr[indices]

arr = np.array([3, 1, 4, 1, 5, 9, 2, 6])
indices, values = top_k_elements(arr, 3)
print(f"Top 3 values: {values} at indices {indices}")

# Matrix chain multiplication (dynamic programming)
def matrix_chain_order(dims):
    """
    Find optimal parenthesization for matrix multiplication
    dims: array of dimensions [p0, p1, ..., pn] for matrices A1(p0×p1), A2(p1×p2), ...
    """
    n = len(dims) - 1
    m = np.zeros((n, n), dtype=int)

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            m[i, j] = float('inf')

            for k in range(i, j):
                cost = m[i, k] + m[k + 1, j] + dims[i] * dims[k + 1] * dims[j + 1]
                if cost < m[i, j]:
                    m[i, j] = cost

    return m[0, n - 1]

# Example: matrices 10×30, 30×5, 5×60

```

```

dims = np.array([10, 30, 5, 60])
print(f"Minimum multiplications: {matrix_chain_order(dims)}")

```

Advanced NumPy for Optimization

```

# Vectorized operations instead of loops
def sum_of_squares_slow(n):
    """Using Python loop - slow"""
    total = 0
    for i in range(n):
        total += i ** 2
    return total

def sum_of_squares_fast(n):
    """Using NumPy - fast"""
    arr = np.arange(n)
    return np.sum(arr ** 2)

# Broadcasting
a = np.array([[1, 2, 3]])          # (1, 3)
b = np.array([[1], [2], [3]])     # (3, 1)
c = a + b                          # (3, 3) - broadcasting in action

# Fancy indexing for permutations
def apply_permutation(arr, perm):
    """Apply permutation to array"""
    return arr[perm]

arr = np.array([10, 20, 30, 40, 50])
perm = np.array([4, 0, 2, 1, 3])
print(apply_permutation(arr, perm)) # [50, 10, 30, 20, 40]

# Memory-efficient operations with views
large_array = np.arange(1000000)
view = large_array[::2] # Every other element - no copy!
view[:] = 0             # Modifies original array

# Random sampling for algorithms
def reservoir_sampling(stream, k):
    """Reservoir sampling using NumPy"""
    reservoir = np.zeros(k, dtype=int)

    for i, item in enumerate(stream):
        if i < k:
            reservoir[i] = item
        else:
            j = np.random.randint(0, i + 1)
            if j < k:
                reservoir[j] = item

    return reservoir

stream = range(100)
sample = reservoir_sampling(stream, 10)
print(f"Random sample of 10: {sample}")

```


Using Pandas for Data Structures

Pandas provides high-level data structures for data manipulation.

Installation

```
pixi add pandas
```

Series and DataFrame Basics

```
import pandas as pd

# Series - 1D labeled array
s = pd.Series([1, 2, 3, 4, 5], index=['a', 'b', 'c', 'd', 'e'])
print(s)
print(s['b'])          # Access by label
print(s[1])            # Access by position

# DataFrame - 2D labeled data structure
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [25, 30, 35, 40],
    'city': ['NYC', 'LA', 'Chicago', 'Houston']
}
df = pd.DataFrame(data)
print(df)

# Creating from different sources
df_from_dict = pd.DataFrame([
    {'a': 1, 'b': 2},
    {'a': 3, 'b': 4, 'c': 5}
])

df_from_array = pd.DataFrame(
    np.random.randn(4, 3),
    columns=['A', 'B', 'C'],
    index=['row1', 'row2', 'row3', 'row4']
)

# DataFrame attributes
print(df.shape)        # (4, 3)
print(df.columns)       # ['name', 'age', 'city']
print(df.index)         # RangeIndex(0, 4)
print(df.dtypes)        # Data types of columns
print(df.info())        # Summary
print(df.describe())    # Statistics
```

Indexing and Selection

```
# Column selection
print(df['name'])          # Single column (Series)
print(df[['name', 'age']]) # Multiple columns (DataFrame)

# Row selection
print(df.iloc[0])          # By position
print(df.loc[0])           # By label
print(df.iloc[1:3])        # Slicing by position
print(df.head(2))          # First 2 rows
print(df.tail(2))          # Last 2 rows

# Boolean indexing
print(df[df['age'] > 30])
print(df[(df['age'] > 25) & (df['city'] == 'NYC')])

# Setting values
df.loc[0, 'age'] = 26
df['salary'] = [50000, 60000, 70000, 80000]

# Conditional selection with query
result = df.query('age > 30 and city == "Chicago"')
```

Data Manipulation

```
# Sorting
df_sorted = df.sort_values('age')
df_sorted_desc = df.sort_values('age', ascending=False)
df_multi_sort = df.sort_values(['city', 'age'])

# Adding/removing columns
df['age_group'] = df['age'].apply(lambda x: 'young' if x < 30 else 'old')
df.drop('age_group', axis=1, inplace=True)

# Applying functions
df['age_squared'] = df['age'].apply(lambda x: x ** 2)
df['name_upper'] = df['name'].str.upper()

# Grouping and aggregation
grouped = df.groupby('city')['age'].mean()
multi_agg = df.groupby('city').agg({
    'age': ['mean', 'min', 'max'],
    'salary': 'sum'
})

# Pivot tables
pivot = df.pivot_table(
    values='salary',
    index='city',
    aggfunc='mean'
)

# Merging and joining
df1 = pd.DataFrame({
    'key': ['A', 'B', 'C'],
    'value1': [1, 2, 3]
})
df2 = pd.DataFrame({
    'key': ['A', 'B', 'D'],
    'value2': [4, 5, 6]
})

merged_inner = pd.merge(df1, df2, on='key', how='inner')
merged_outer = pd.merge(df1, df2, on='key', how='outer')
merged_left = pd.merge(df1, df2, on='key', how='left')

# Concatenation
df_concat = pd.concat([df1, df2], ignore_index=True)
df_concat_cols = pd.concat([df1, df2], axis=1)
```

Time Series and Window Functions

```
# Creating time series
dates = pd.date_range('2024-01-01', periods=100, freq='D')
ts = pd.Series(np.random.randn(100), index=dates)

# Resampling
monthly = ts.resample('M').mean()
weekly = ts.resample('W').sum()

# Rolling window operations
rolling_mean = ts.rolling(window=7).mean()
rolling_std = ts.rolling(window=7).std()

# Expanding window
expanding_max = ts.expanding().max()

# Shift operations
ts_shifted = ts.shift(1)          # Lag
ts_pct_change = ts.pct_change()  # Percent change
```

Advanced Data Structures with Pandas

```
# Multi-index DataFrame
arrays = [
    ['A', 'A', 'B', 'B'],
    [1, 2, 1, 2]
]
index = pd.MultiIndex.from_arrays(arrays, names=['first', 'second'])
df_multi = pd.DataFrame(np.random.randn(4, 2), index=index, columns=['X', 'Y'])
print(df_multi)

# Accessing multi-index
print(df_multi.loc['A'])
print(df_multi.loc[('A', 1)])

# Categorical data (efficient for repeated values)
df['city_cat'] = df['city'].astype('category')
print(df['city_cat'].cat.categories)

# Using pandas for graph adjacency
edges = pd.DataFrame({
    'source': ['A', 'A', 'B', 'C'],
    'target': ['B', 'C', 'C', 'D'],
    'weight': [1, 2, 3, 4]
})

# Convert to adjacency dictionary
def edges_to_graph(edges_df):
    graph = {}
    for _, row in edges_df.iterrows():
        source, target, weight = row['source'], row['target'], row['weight']
        if source not in graph:
            graph[source] = []
        graph[source].append((target, weight))
    return graph

graph = edges_to_graph(edges)
print(graph)
```

Performance Optimization

```
# Vectorized operations (fast)
df['result'] = df['age'] * df['salary'] / 1000

# Avoid iterrows (slow)
# BAD:
# for idx, row in df.iterrows():
#     df.loc[idx, 'result'] = row['age'] * row['salary']

# Use apply (better)
df['result'] = df.apply(lambda row: row['age'] * row['salary'], axis=1)

# Use vectorized operations (best)
df['result'] = df['age'] * df['salary']

# Reading large files efficiently
# chunksize for large files
chunk_iter = pd.read_csv('large_file.csv', chunksize=10000)
for chunk in chunk_iter:
    process(chunk)

# Memory optimization
df_optimized = df.copy()
for col in df_optimized.select_dtypes(include=['int64']).columns:
    df_optimized[col] = df_optimized[col].astype('int32')

# Using eval for complex expressions
df.eval('result = age * salary / 1000', inplace=True)
```

Practical Algorithm Examples with Pandas

```
# Finding duplicates
duplicates = df[df.duplicated(subset=['name'])]
df_deduped = df.drop_duplicates(subset=['name'])

# Value counts (frequency counting)
city_counts = df['city'].value_counts()

# Binning continuous data
df['age_bin'] = pd.cut(df['age'], bins=[0, 30, 40, 100], labels=['young', 'middle', 'old'])

# Computing ranks
df['age_rank'] = df['age'].rank(method='dense')

# Cross tabulation
crosstab = pd.crosstab(df['city'], df['age_bin'])

# Window functions for running calculations
df_sorted = df.sort_values('age')
df_sorted['cumulative_salary'] = df_sorted['salary'].cumsum()
df_sorted['rank'] = df_sorted['salary'].rank(ascending=False)

# Efficient string operations
df['name_length'] = df['name'].str.len()
df['contains_a'] = df['name'].str.contains('a', case=False)
df['name_split'] = df['name'].str.split('').str[0]

# Memory-efficient data types
df_efficient = pd.DataFrame({
    'category': pd.Categorical(['A', 'B', 'A', 'C']),
    'value': pd.array([1, 2, 3, 4], dtype='Int64'), # Nullable integer
    'flag': pd.array([True, False, None, True], dtype='boolean') # Nullable boolean
})
```

Recommended Books

Beginner to Intermediate

1. **“Python Data Structures and Algorithms” by Benjamin Baka**
 - Practical approach with Python examples
 - Covers fundamental to advanced topics
2. **“Grokking Algorithms” by Aditya Bhargava**
 - Visual and intuitive explanations
 - Great for beginners
 - Language-agnostic but easy to translate to Python
3. **“Problem Solving with Algorithms and Data Structures using Python” by Brad Miller and David Ranum**
 - Freely available online
 - Strong focus on Python implementation

Advanced

1. **“Introduction to Algorithms” (CLRS) by Cormen, Leiserson, Rivest, and Stein**
 - The definitive reference
 - Pseudocode (not Python-specific)
 - Very comprehensive and rigorous
2. **“The Algorithm Design Manual” by Steven Skiena**
 - Practical approach
 - Great for interview preparation
 - War stories from real applications
3. **“Algorithms” by Robert Sedgewick and Kevin Wayne**
 - Thorough coverage
 - Available with Java examples, but concepts translate well

Interview Preparation

1. **“Cracking the Coding Interview” by Gayle Laakmann McDowell**
 - Essential for technical interviews
 - 189 programming questions with solutions
2. **“Elements of Programming Interviews in Python” by Adnan Aziz, Tsung-Hsien Lee, and Amit Prakash**
 - Python-specific interview preparation
 - Challenging problems with detailed solutions

Online Resources

- **LeetCode** - Practice problems
- **HackerRank** - Structured learning paths
- **Project Euler** - Mathematical/computational problems
- **GeeksforGeeks** - Theory and examples
- **Real Python** - Python-specific tutorials

Practice Projects

1. Implement Your Own Collections Module

Create implementations of:

- OrderedDict
- Counter
- DefaultDict
- ChainMap

2. Build a Mini Database

Implement:

- B-Tree for indexing
- Hash table for quick lookups
- Query parser
- Transaction support

3. Graph Algorithms Visualizer

Create a tool that:

- Visualizes graph traversals
- Shows algorithm execution step-by-step
- Supports different graph types

4. Algorithm Performance Analyzer

Build a tool that:

- Times different sorting algorithms
- Plots performance curves
- Compares theoretical vs actual complexity

5. LRU Cache Implementation

Implement an LRU cache using:

- Hash map for $O(1)$ lookup
- Doubly linked list for $O(1)$ eviction
- Support for TTL (time-to-live)

Conclusion

This guide covers the essential data structures and algorithms you need for:

- Technical interviews
- Competitive programming
- Real-world software development
- Understanding algorithm complexity

Key Takeaways:

1. Master the fundamentals before advanced topics
2. Practice implementing algorithms from scratch
3. Understand time and space complexity
4. Use Python's built-in data structures when appropriate
5. Leverage NumPy and Pandas for performance-critical code
6. Solve problems on platforms like LeetCode regularly

Next Steps:

1. Work through the examples in this guide
2. Implement each data structure yourself
3. Solve 2-3 algorithm problems daily
4. Read one of the recommended books
5. Build a project that uses multiple concepts

Happy coding! 🚀